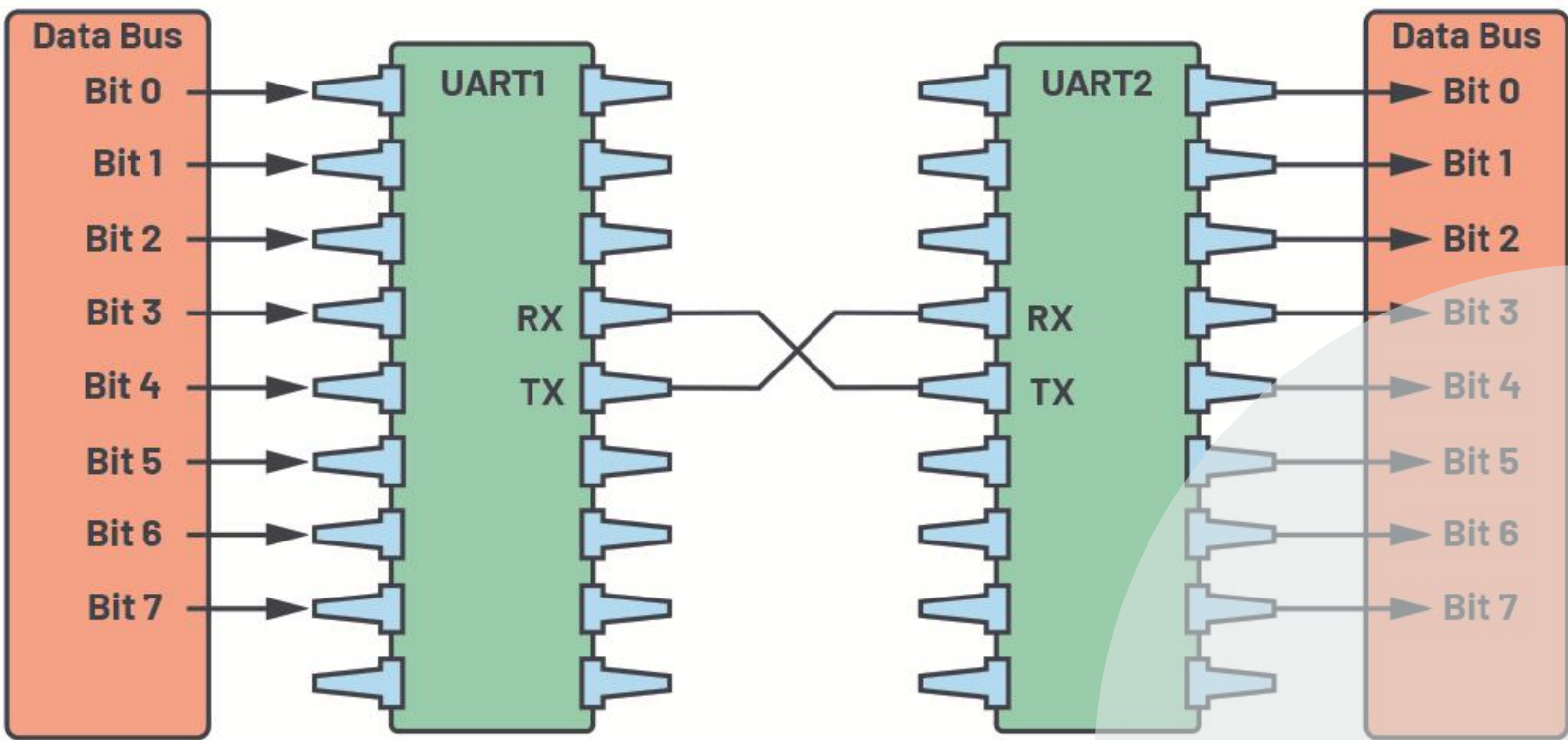




UART

Prepared by:

Arwa Ashraf Kantoush

TABLE OF CONTENTS

1. INTRODUCTION & OVERVIEW

- 1.1 Project Overview & Objectives
- 1.2 UART_RX Architecture Summary
- 1.3 UART_TX Architecture Summary
- 1.4 Operational Features & Frame Transmission Sequence

2. RTL DESIGN & MODELING

2.1 TRANSMITTER

- 2.1.1 Parity Calculator
- 2.1.2 Serializer Shift Register
- 2.1.3 MUX
- 2.1.4 FSM_TX Controller
- 2.1.5 UART_TX Top Integration

2.2 RECEIVER

- 2.2.1 Parity Checker
- 2.2.2 Deserializer Shift Register
- 2.2.3 FSM_RX Controller & Start-Bit Detection
- 2.2.4 UART_RX Top Integration

3. RTL QUALITY CHECK (LINTING)

4. SIMULATION & AUTOMATION

5. ELABORATION FLOW

6. SYNTHESIS FLOW

7. IMPLEMENTATION FLOW (PLACE
& ROUTE)

1. INTRODUCTION & OVERVIEW

1.1 Project Overview & Objectives

This project implements a fully parameterized, cycle-accurate UART core in synthesizable SystemVerilog. The architecture provides independent transmitter (`uart_tx`) and receiver (`uart_rx`) modules featuring modular RTL partitioning, dynamic parity configuration, deterministic FSM control, and error detection.

1.2 UART_RX Architecture Summary

The receiver (`uart_rx`) detects frame boundaries, reconstructs parallel data, and verifies integrity. It integrates a SIPO Deserializer, an FSM_RX controller, and a Parity_Checker. Using a "flag-and-deliver" approach, it outputs the received byte on `o_data` while pulsing `o_valid`, `o_parity_err`, and `o_frame_err` for one clock cycle upon completion.

1.3 UART_TX Architecture Summary

The transmitter (`uart_tx`) converts parallel input into a standard serial bitstream on `o_tx`. Composed of an FSM_TX controller, a PISO Serializer, a Parity_Calculator, and an output MUX, it holds `o_busy` high during transmission, blocks overlapping write attempts, and idles high.

1.4 Operational Features & Frame Transmission Sequence

Each bit period spans exactly one system clock cycle (no oversampling). The transmission frame structure is:

- **Idle:** Serial line held high (`1'b1`).
- **Start Bit:** Driven low (`1'b0`) for one cycle.
- **Data Payload:** `DATA_W` bits (default 8) transmitted LSB-first.
- **Parity Bit (Optional):** Appended when enabled (`i_par_en = 1`), supporting even or odd modes.
- **Stop Bit:** Driven high (`1'b1`) for one cycle.

2. RTL DESIGN & MODELING

2.1 TRANSMITTER

2.1.1 Parity Calculator

Computes and registers the parity bit dynamically upon a valid, non-busy request (`i_valid && i_par_en && !i_busy`) using the reduction operation: `o_parity_bit <= (^i_data) ^ i_par_odd`. The value remains latched throughout transmission.

2.1.2 Serializer Shift Register

A PISO shift register of width `DATA_W`. It captures `i_data` during load and shifts rightward (LSB-first) on each active `i_ser_en` cycle. A dedicated counter tracks shifted bits and asserts `o_ser_done` concurrently with the final bit.

2.1.3 MUX

A 4-to-1 multiplexer driven by `i_sel` to route the appropriate bit to `o_tx`: Start bit (2'b00), Serial Data (2'b01), Parity bit (2'b10), or Stop bit/Idle (2'b11).

2.1.4 FSM_TX Controller

A Moore FSM traversing `IDLE`, `START`, `DATA`, `PARITY`, and `STOP`. It manages line selection (`o_sel`), shift enable (`o_ser_en`), and `o_busy`. It branches past `PARITY` directly to `STOP` when parity is disabled and rejects write attempts while busy.

2.1.5 UART_TX Top Integration

The top module structuralizes the transmitter by interconnecting the FSM, serializer, parity calculator, and multiplexer into a self-contained entity with clean input/output boundaries.

2.2 RECEIVER

2.2.1 Parity Checker

Combinational logic comparing the received parity bit against the computed reduction of the deserialized data:

$$o_parity_err = i_par_en \wedge (i_parity_bit \neq ((\text{textasciicircum}$$

It is clamped to zero when parity is disabled.

2.2.2 Deserializer Shift Register

A SIPO register that shifts incoming `i_rx` bits into the MSB position on `i_shift_en`. An internal counter signals `o_shift_done` once `DATA_W` bits are assembled, resetting cleanly to prevent phase shift on subsequent frames.

2.2.3 FSM_RX Controller & Start-Bit Detection

Governs reception across IDLE, DATA, PARITY, STOP, and PULSE.

- **Start Detection:** Samples `i_rx` only while in IDLE (!`o_busy`), triggering active reception on a low level while masking false start bits during data transfer.
- **Status & Delivery:** Latches parity during PARITY, checks for a valid high stop bit during STOP (registering framing errors if low), and asserts single-cycle pulses on `o_valid` and active error flags during PULSE.

2.2.4 UART_RX Top Integration

Integrates the deserializer, parity checker, and FSM. It coordinates timing so that `o_data` remains stable while `o_valid`, `o_parity_err`, and `o_frame_err` pulse simultaneously.

3. RTL QUALITY CHECK (LINTING)

RX

Questa Lint 2021.1 (C:/Users/Arwa/Desktop/Digital_IC/NTI_FPGA/UART_sv/uart_rx.lint.db)

File Edit View Schematic Window Help

Flow Navigator

UART_RX

```
1 module u_uart_rx #(
2     parameter DATA_W = 8
3 )
4 (
5     input logic i_clk,
6     input logic i_rst_n,
7     input logic i_rx,
8     input logic i_par_odd,
9     input logic i_par_en,
10    output logic [DATA_W-1:0] o_data,
11    output logic o_parity_err,
12    output logic o_valid,
13    output logic o_busy,
14    output logic o_frame_err
15 );
16 logic shift_en;
17 logic shift_done;
18 logic parity_bit;
19
20 Deserializer #(DATA_W(DATA_W)) u_Deserializer (
21     .i_clk(i_clk),
22     .i_rst_n(i_rst_n),
23     .i_rx(i_rx),
24     .i_shift_en(shift_en),
25     .i_shift_done(shift_done),
26     .o_data(o_data),
27     .o_parity_err(o_parity_err),
28     .o_valid(o_valid),
29     .o_busy(o_busy),
30     .o_frame_err(o_frame_err)
31 );
32 FSM_RX #(DATA_W(DATA_W)) u_FSM_RX (
33     .i_clk(i_clk),
34     .i_rst_n(i_rst_n),
35     .i_rx(i_rx),
36     .i_par_en(i_par_en),
37     .i_par_odd(i_par_odd),
38     .i_valid(i_valid),
39     .i_shift_en(shift_en),
40     .i_shift_done(shift_done),
41     .o_data(o_data),
42     .o_parity_err(o_parity_err),
43     .o_valid(o_valid),
44     .o_busy(o_busy),
45     .o_frame_err(o_frame_err)
46 );
47 Parity_Checker #(DATA_W(DATA_W)) u_Parity_Checker (
48     .i_data(o_data),
49     .i_parity_err(o_parity_err),
50     .i_valid(o_valid),
51     .i_busy(o_busy),
52     .i_frame_err(o_frame_err)
53 );
```

Schematic 1

Lint Checks

Filter: Type here

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
Warning	Fixed	parameter_name_duplicate		Same parameter name is used in more than one mod...	Deserializer	Nomenclature...	resolved	unassign...	1.1.4.3, 3.2.2.2
Warning	Fixed	always_signal_assign_large		Always block has more signal assignments than the s...	FSM_RX	Rtl Design Style	resolved	unassign...	2.6.1.3

Transcript Message Viewer Lint Checks Design Metrics Design Information Status History Lint Dashboard

C:/Users/Arwa/Desktop/Digital_IC/NTI_FPGA/UART_sv/uart_rx.sv [uart_rx]

TX

Questa Lint 2021.1 (C:/Users/Arwa/Desktop/Digital_IC/NTI_FPGA/UART_sv/uart_tx.lint.db)

File Edit View Schematic Window Help

Design

Search: Type Search...

Instance Module

uart_tx (4) uart_tx

```
1 module u_uart_tx #(
2     parameter DATA_W = 8
3 )
4 (
5     input logic i_clk,
6     input logic i_rst_n,
7     input logic [DATA_W-1:0] i_data,
8     input logic i_valid,
9     input logic i_par_en,
10    input logic i_par_odd,
11    output logic o_tx,
12    output logic o_busy
13 );
14 logic PARITY_BIT;
15 logic SER_EN;
16 logic SER_DATA;
17 logic SER_DONE;
18 logic [1:0] SEL;
19
20 Parity_Calculator #(DATA_W(DATA_W)) u_Par
21     .i_clk(i_clk),
22     .i_rst_n(i_rst_n),
23     .i_data(i_data),
24     .i_valid(i_valid),
25     .i_par_en(i_par_en),
26     .i_par_odd(i_par_odd),
27     .o_parity_bit(PARITY_BIT),
28     .o_busy(o_busy)
29 );
30 FSM_TX #(DATA_W(DATA_W)) u_FSM_TX (
31     .i_clk(i_clk),
32     .i_rst_n(i_rst_n),
33     .i_data(i_data),
34     .i_valid(i_valid),
35     .i_par_en(i_par_en),
36     .i_par_odd(i_par_odd),
37     .i_ser_en(SER_EN),
38     .i_ser_data(SER_DATA),
39     .i_ser_done(SER_DONE),
40     .o_tx(o_tx),
41     .o_busy(o_busy)
42 );
43 Serializer #(DATA_W(DATA_W)) u_Serializer (
44     .i_clk(i_clk),
45     .i_rst_n(i_rst_n),
46     .i_data(i_data),
47     .i_valid(i_valid),
48     .i_ser_en(SER_EN),
49     .i_ser_data(SER_DATA),
50     .i_ser_done(SER_DONE),
51     .o_tx(o_tx),
52     .o_busy(o_busy)
53 );
44 MUX #(DATA_W(DATA_W)) u_MUX (
45     .i_data(o_tx),
46     .i_ser_data(SER_DATA),
47     .i_ser_done(SER_DONE),
48     .o_tx(o_tx),
49     .o_busy(o_busy)
50 );
```

Schematic 1

Lint Checks

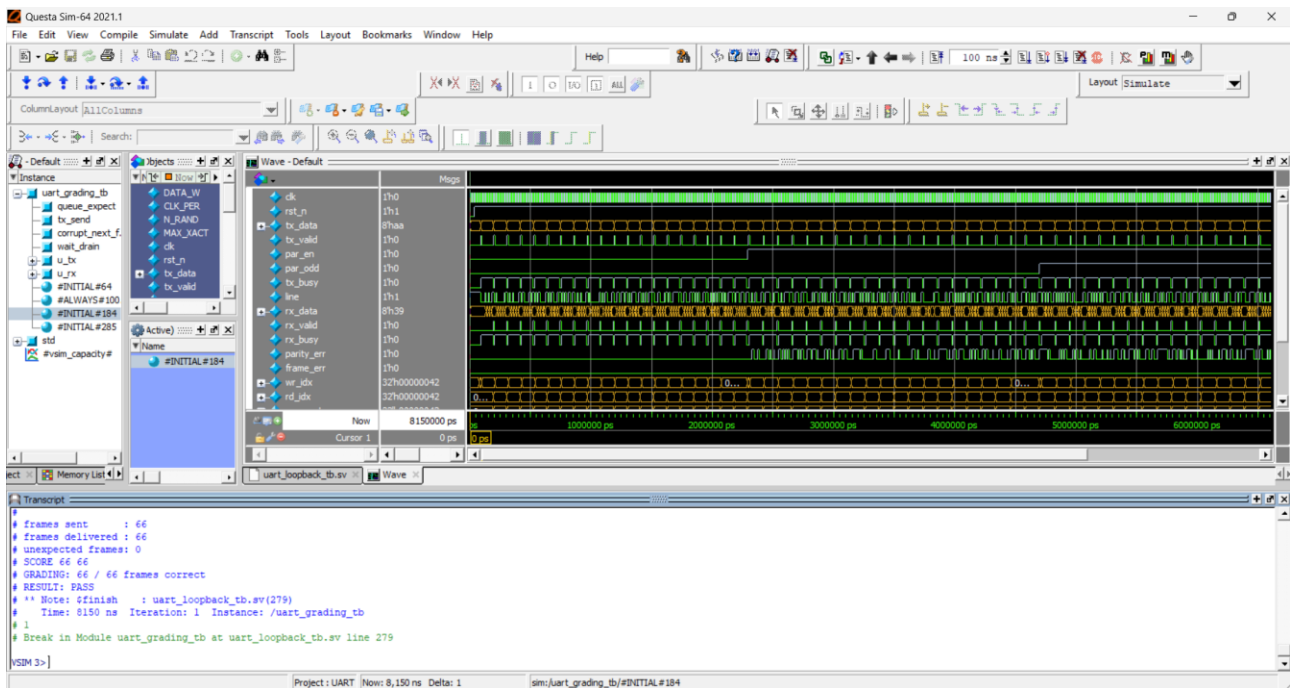
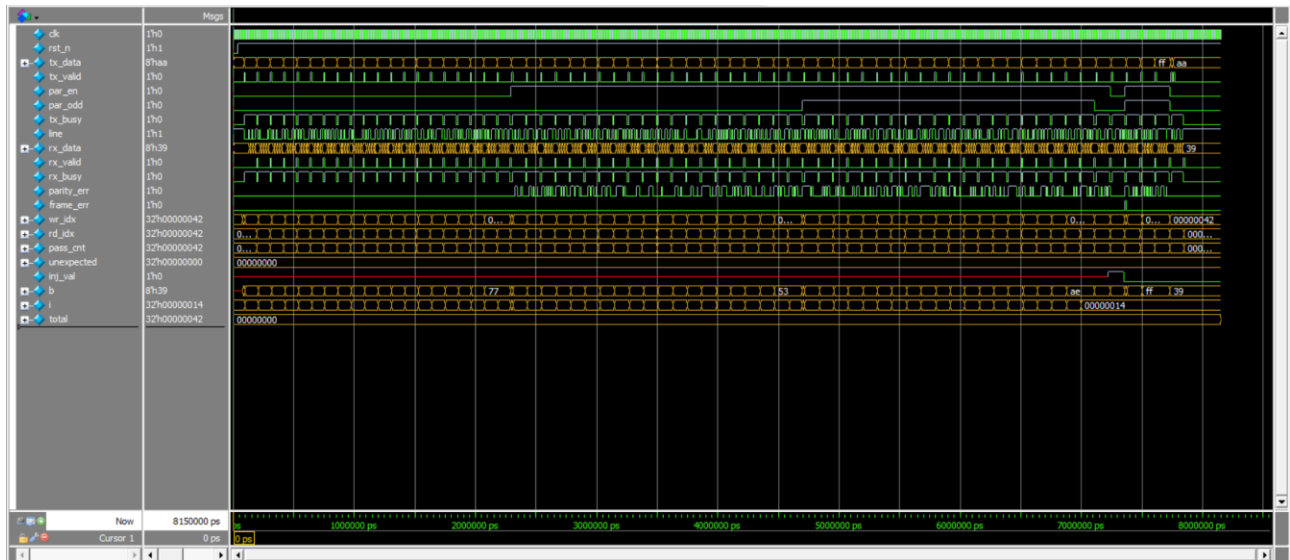
Filter: Type here

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
Warning	Fixed	parameter_name_duplicate		Same parameter name is used in more than one mod...	Parity_Calcul...	Nomenclature...	resolved	unassign...	1.1.4.3, 3.2.2.2

Transcript Message Viewer Lint Checks Design Metrics Design Information Status History Lint Dashboard

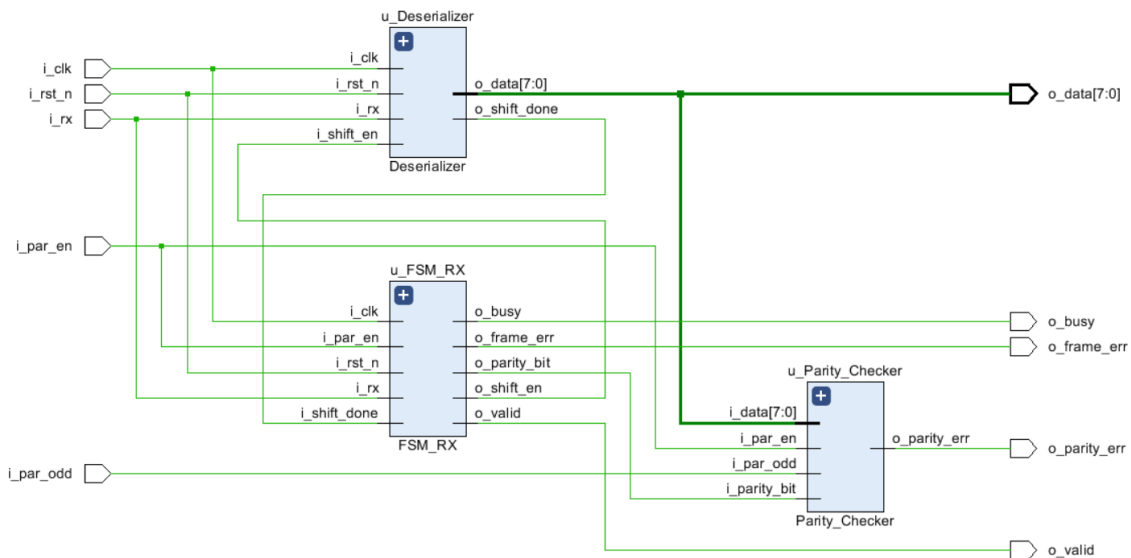
...Users/Arwa/Desktop/Digital_IC/NTI_FPGA/UART_sv/uart_tx.sv [uart_tx]

4. SIMULATION & AUTOMATION

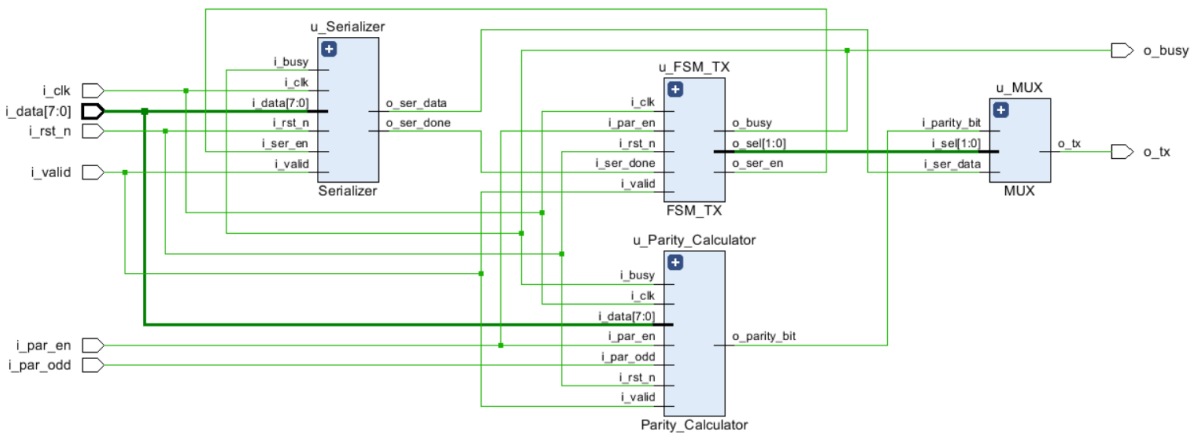


5. ELABORATION FLOW

RX

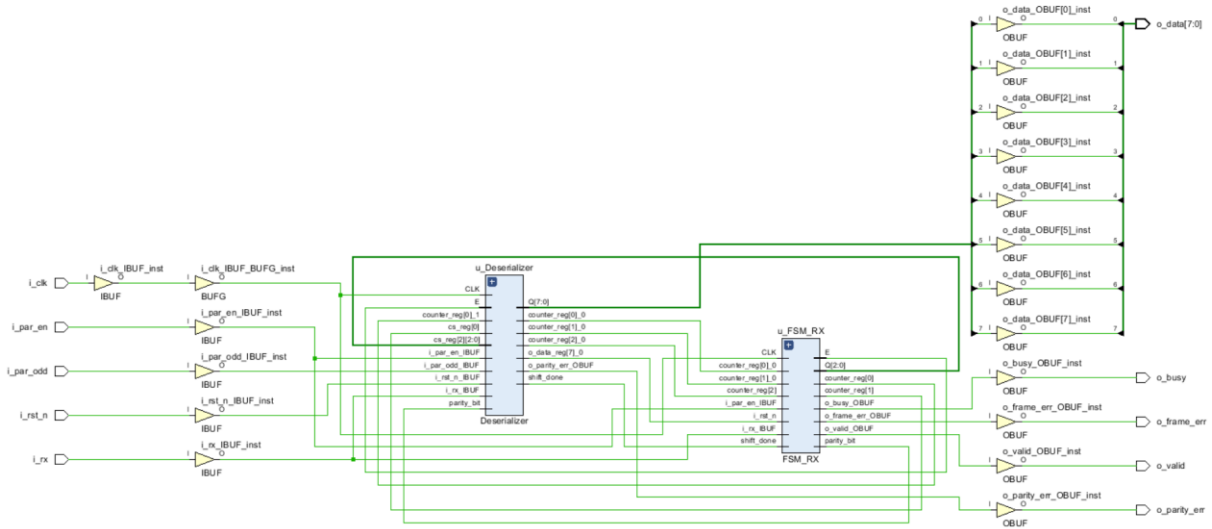


TX

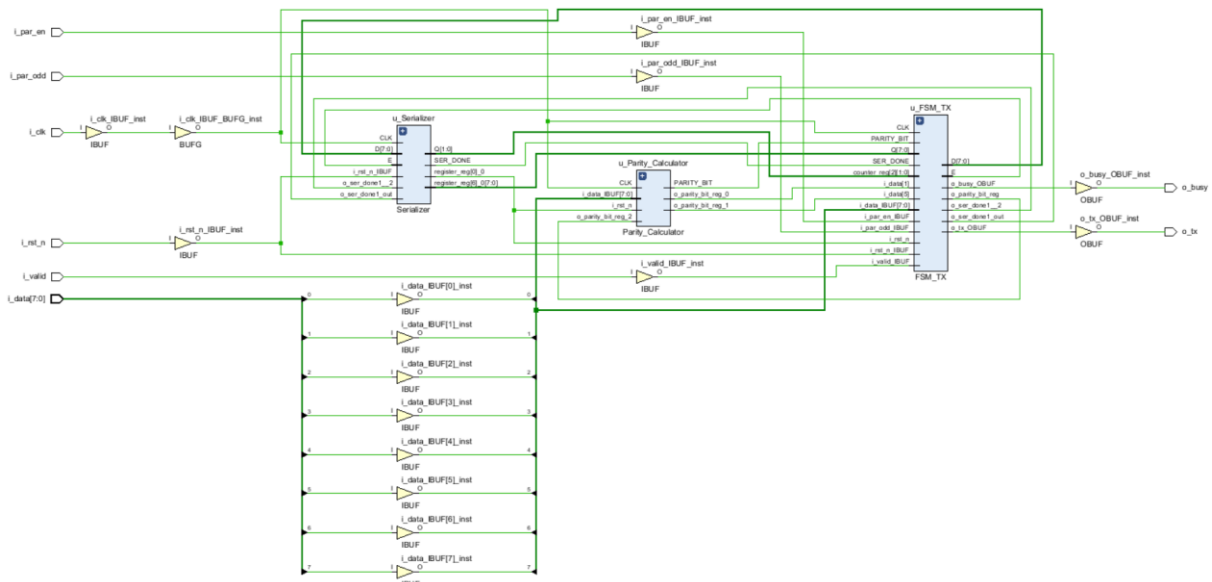


6. SYNTHESIS FLOW

RX



TX



7. IMPLEMENTATION FLOW (PLACE & ROUTE)

RX

